

Zuko

A privacy firewall and fiery little guardian for AI coding agents

This guide covers	
1. What Zuko can do	Every capability, in plain words, with the numbers we measured.
2. Setup	Everything to install and switch on, step by step, with commands.
3. Demo for judges	Ready-to-run demos with the exact prompts, commands and what should appear.

Built for Windows 10/11 · Works with Claude Code (CLI and VS Code), ChatGPT, claude.ai and DeepSeek

Repository: github.com/AuvroIslam/Zuko

Zuko's on-screen character is an unofficial fan tribute to Prince Zuko from *Avatar: The Last Airbender*. Zuko is not affiliated with or endorsed by Viacom International / Nickelodeon. The app is built on the MIT-licensed source code of Coucou (see NOTICE.md).

1. What Zuko can do

AI coding agents such as Claude Code read your files, run commands and send everything they see to a cloud model. Zuko sits in between. It runs on your computer, watches every message and every action, and steps in when something sensitive or dangerous is about to happen. It solves four problems:

Problem	What Zuko does about it
You press “Yes” on agent prompts without reading them.	Scores every action, explains it in plain English and only asks when it matters. Risky actions need a deliberate hold.
Secrets leak into the chat (API keys, passwords, personal data).	Replaces them with placeholders before they leave your PC, then puts the real values back locally.
You can't set rules the agent must follow.	A policy you control: blocked websites, folders, commands and tools — enforced on every action.
You need answers about private data without sharing it.	The model works with placeholders and a safe description; the real value is used only on your machine.

1.1 Secrets never reach the cloud AI

You type: “My API key is `sk-proj-...`, put it in `.env`”. The cloud model receives “My API key is `{{API_KEY_1}}`, put it in `.env`” plus a note that `API_KEY_1` is an OpenAI API key. It answers by writing `OPENAI_API_KEY={{API_KEY_1}}`, and Zuko swaps the real key in on your computer just before the file is written. Your `.env` works; the provider never saw the key.

- **Detects** 51 secret formats (OpenAI, Anthropic, AWS, GitHub, Stripe, Google, Slack, Hugging Face, JWTs, private keys, database connection strings, passwords ...) and personal data: emails, phone numbers (including Bangladesh +880), payment cards (Luhn-checked), IBANs (checksum-checked), national IDs (with context words), optional public IPs, and your own custom words.
- **Stable placeholders:** the same value always gets the same placeholder, so long conversations stay consistent.
- **Gateway mode (complete):** Claude Code talks to the cloud through Zuko's local gateway, so prompts, files the agent reads, command output, `@file` attachments and IDE selections are all masked. Works with a Claude Pro/Max login or an API key.
- **Hooks-only mode (fallback):** no gateway; a prompt that contains a secret is stopped before it is sent, and a masked copy is offered.
- **Safe by design:** real values are filled back in only for local file writes and local shell commands. A command that would send a secret over the internet is asked or blocked, so a tricked AI cannot use Zuko to leak it.

1.2 Agent firewall — rules the agent can't break

- **Your policy:** blocked and allowed domains, blocked read/write folders, sensitive files, blocked or “always ask” commands and tools (including MCP).
- **Checks every tool call:** Read, Write, Edit, WebFetch, WebSearch, Bash, PowerShell and MCP tools. Shell commands are parsed for network access, deletes, privilege changes, package installs, pipe-to-shell and obfuscation.
- **Chain rules** catch multi-step leaks: `read .env` → send it with `curl` (even base64/hex/URL-encoded) is blocked and the reason names the step that read it. Others: tainted egress, running downloaded code, unknown MCP tools, unanalyzable commands.
- **Self-protection:** the agent cannot edit Zuko's policy, Claude Code's settings, Zuko's binaries, or kill Zuko.

- **Works even when the app is closed:** the hook relay carries the same policy engine and still blocks what you banned.
- **Defense in depth:** blocked paths and domains can be copied into Claude Code's own deny rules.
- **Per-project rules:** a project can add `.zuko/policy.json`, which can only make rules stricter. Enforce or Monitor mode.

1.3 Approvals you actually read

- Every action gets a **risk tier** (low · medium · high · critical), a score, a headline that leads with the consequence ("DELETES the folder dist/ and everything in it") and impact chips (undo, scope, network, data).
- **Low risk** passes automatically · **medium** gets a normal card · **high** needs **hold-to-allow** · **critical** or rule-breaking is blocked.
- **Rubber-stamp detection:** approving risky actions faster than anyone could read them makes Zuko slow you down.

1.4 Monitoring and records

- **Live activity feed** on the island and in Settings → Activity (allowed, asked, blocked, masked).
- **Tamper-evident audit log:** every decision is a hash-chained record; one click verifies the chain. It never stores secret values.
- **Encrypted vault:** masked values are stored encrypted (XChaCha20-Poly1305) with the key in Windows Credential Manager.
- **Vault tools:** Inspect (safe facts such as card brand + last 4 or phone country — never the value), Copy value (clipboard clears after 30 s), Reveal (explicit click, hides after 10 s), Forget.
- **Open file:** rows for files the agent wrote open the file in VS Code, for edits you want to make by hand.

1.5 Documents, clipboard and chat

- **Drop a file** (.txt, .md, .pdf, code, .env, json, .csv ...) on Zuko or in Settings → Documents: you get a masked copy (name `.zuko.md`) that is safe to upload anywhere. PDFs are read from their text layer.
- **Clipboard hotkeys:** **Ctrl+Alt+M** masks the text on your clipboard, **Ctrl+Alt+U** restores it — works with any chat app.
- **Island chat** with your choice of **Claude**, **OpenAI** or a free **local model (Ollama)**. Cloud providers only ever receive masked text and answers are restored before you read them; with Ollama nothing leaves the PC.

1.6 Browser extension (ChatGPT, claude.ai, DeepSeek)

- Masks prompts and uploaded files before they are sent; a tripwire checks every outgoing request for known secrets.
- Restores real values in the answers you read and in copied code; **Alt+R** toggles between real values and placeholders.
- Links to the desktop app so one vault and one policy cover both Claude Code and web chats; works on its own when the app is closed.

1.7 Optional local AI (Ollama)

- Off by default. When on, a local model (default `gemma3:4b`) finds what patterns miss — person names, street addresses, company names, unknown secret formats — and writes plain-English explanations on risky approval cards.
- **It can only make Zuko stricter:** it adds masks, never removes them, never approves anything; every value it reports must appear word-for-word; it only sees text that is already masked and only talks to Ollama on your own PC.

1.8 Zuko the mascot

A chibi firebender lives at the top of your screen and reacts to what happens: he fire-punches blocked actions, flicks fire at masked secrets, simmers with a flame aura on high-risk approvals, hovers on a ring of fire while the agent works, and drops in with a fire swirl at launch. Triple-click him and he punches fire at your cursor.

1.9 Measured results

Measure	Result
Attack benchmark (51 scenarios: exfiltration chains, prompt injection, encoded secret egress, self-protection, safe workflows, near-misses)	100% attacks caught · 0% false blocks
Decision speed per agent action	≈ 0.3 ms
Scanning 1 MB of text for secrets	≈ 50 ms
Automated tests	449 passing (engine 134 · app 138 · relay 14 · native host 22 · extension 141)
Live end-to-end with the real Claude Code CLI	Provider received only <code>{{API_KEY_1}}</code> ; <code>.env</code> got the real key; <code>pastebin curl</code> denied

1.10 Honest limits

- Pattern detection can miss unknown secret formats and free-form names (custom terms and the optional local AI help).
- Edits that depend on a secret's actual content ("uppercase my password") cannot be done by the cloud AI — use Open file and edit by hand.
- No OS sandbox on native Windows: programs the agent starts (for example an npm install script) are outside Zuko's view.
- The browser extension depends on the chat sites' current page structure, which can change.

2. Setup

Do this once on the demo laptop. Commands are for **Git Bash** unless marked PowerShell.

2.1 What you need

Item	Why	How to get it
Windows 10/11	Zuko's desktop app	—
Node.js 20+ (24 recommended)	Builds the UI and extension	nodejs.org
Rust (rustup)	Builds the app and engine	rustup.rs (see 2.2)
A C toolchain: MinGW-w64 (WinLibs) or MSVC Build Tools	Needed by some Rust crates	winget (see 2.2)
Git	Get the code	git-scm.com
Claude Code (CLI or VS Code extension), logged in	The agent Zuko protects	claude.com/claude-code
Chrome or Edge	Browser extension demo	—
Ollama + gemma3:4b (optional)	Local AI demo, free island chat	ollama.com, then: ollama pull gemma3:4b

Do I need a Claude API key? No. The Claude Code demos need Claude Code logged in with any account — a Claude Pro/Max subscription login works, or an API key. The island chat works with a Claude key, an OpenAI key, or free local Ollama (no key). Everything else — the relay, documents, clipboard, vault, audit log, local AI, the browser extension on ChatGPT/DeepSeek, the benchmark and the mock UI — needs no Claude account at all.

2.2 Install the toolchain (once)

PowerShell:

```
winget install --id BrechtSanders.WinLibs.POSIX.UCRT -e --scope user
# Rust with the GNU toolchain (no Visual Studio needed):
# download rustup-init.exe from https://rustup.rs and run:
rustup-init.exe -y --default-host x86_64-pc-windows-gnu
rustup target add wasm32-unknown-unknown
# put cargo on PATH permanently, then OPEN A NEW TERMINAL:
$userPath = [Environment]::GetEnvironmentVariable("Path", "User")
[Environment]::SetEnvironmentVariable("Path", "$env:USERPROFILE\.cargo\bin;$userPath", "User")
```

If you already use Visual Studio Build Tools ("Desktop development with C++"), the default MSVC Rust toolchain works too.

Git Bash (if cargo is still "not found", add it for this shell and in ~/.bashrc):

```
echo 'export PATH="$HOME/.cargo/bin:$PATH"' >> ~/.bashrc
export PATH="$HOME/.cargo/bin:$PATH"
cargo --version
```

2.3 Get and run Zuko

```
git clone https://github.com/AuvroIslam/Zuko.git
cd Zuko/app
npm install
npm run tauri dev      # first build takes a few minutes; Zuko appears at the top of the screen
```

Keep this terminal open — **Ctrl+C** stops Zuko. To get a normal installed app instead, run `npm run pack` and use the installer in `app/release/`.

2.4 Switch protection on

1. Click the **gear** on the island (or right-click the tray icon → **Settings...**), open **Protection**.
2. Turn on **Hooks** (firewall + approvals) and **Gateway** (complete secret masking). Optionally **Deny rules**.
3. Click **Review changes...** — you see the exact diff for `~/.claude/settings.json` and the backup file name — then **Apply**.
4. **Restart Claude Code** (close and reopen the terminal session or VS Code panel). The status dots in Settings → Protection turn green.

Important: with Gateway on, Claude Code reaches the cloud through Zuko. If Zuko is closed, Claude Code cannot connect. Before quitting Zuko for good: Settings → Protection → turn Gateway off → Review → Apply → restart Claude Code.

2.5 Island chat provider (optional)

Settings → **Chat** → choose **Claude**, **OpenAI** or **Local (Ollama)**. For Claude or OpenAI paste the API key (stored in Windows Credential Manager) and pick a model; for Ollama no key is needed — just have Ollama running with `gemma3:4b`. Switching provider starts a new conversation.

2.6 Your policy (optional)

Settings → **Policy** ships with sensible defaults (for example `pastebin.com`, `webhook.site` and `*.ngrok.io` blocked; `~/ssh` blocked from reading; `git push --force*` blocked). Add your own websites, folders, commands and custom words (client names, codenames), then **Save policy**.

2.7 Browser extension

```
cd Zuko/app
cargo build -p zuko-core --target wasm32-unknown-unknown --release # engine for the browser
cargo build --release -p zuko-native-host # bridge to the desktop app
cd ../extension
npm install
npm run build # -> extension/dist
# the Zuko app registers the browser bridge itself at launch (Settings -> Browser)
```

Chrome: `chrome://extensions` (Edge: `edge://extensions`) → turn on **Developer mode** → **Load unpacked** → choose `Zuko/extension/dist`. With the Zuko app running, the extension popup shows **Desktop app: linked**.

2.8 Local AI (optional)

```
ollama pull gemma3:4b # once (about 3.3 GB); Ollama must be running
```

Settings → **Local AI** → turn it on → model `gemma3:4b` → **Save** → press **Test**. A scan takes about 7–10 s on a typical laptop; the default timeout is 20 s.

2.9 Everyday use

Task	How
Open the island / settings	Move the mouse to the top edge where the island sits; click Zuko. Gear icon = Settings.
Move the island	Drag its header left or right along the top edge. Double-click the header (or Settings → General → Reset island position) to recentre.
Pause approval cards	Tray icon → Pause protection prompts (rules still apply).
Quit	Tray icon (^ in the taskbar) → right-click Zuko → Quit. Or Ctrl+C in the dev terminal.
Undo everything	Settings → Protection → turn all off → Review changes → Apply. Your old settings come back exactly.

2.10 Troubleshooting

Symptom	Fix
cargo metadata ... program not found	cargo is not on PATH in this terminal: see 2.2, then open a new terminal.
Port 1420 is already in use	An old dev server is still running. Git Bash: kill it (below), then run <code>npm run tauri dev</code> again.
Claude Code cannot connect	Gateway is on but Zuko is not running: start Zuko, or turn Gateway off and restart Claude Code.
Nothing happens in Claude Code	Restart Claude Code after Apply; check Settings → Protection shows Hooks installed.
Extension popup: Desktop app not linked	Keep Zuko running, check Settings → Browser shows the bridge registered (press Register), then reload the extension.

```
taskkill //IM zuko.exe //F
PID=$(netstat -ano | grep ":1420 " | grep LISTENING | awk '{print $5}' | head -1)
[ -n "$PID" ] && taskkill //F //T //PID $PID
```

2.11 Where Zuko keeps its data

Path	What
%APPDATA%\Zuko\policy.json	Your policy
%LOCALAPPDATA%\Zuko\vault.bin	Masked values, encrypted (key in Credential Manager)
%LOCALAPPDATA%\Zuko\audit\audit.jsonl	Hash-chained audit log
%LOCALAPPDATA%\Zuko\inbox\	Dropped files and their masked copies
%LOCALAPPDATA%\Zuko\zuko.log	Diagnostics (never contains secret values)

3. Demo for judges

Each demo lists the exact steps, prompts and commands, and what should appear. Use only the fake values below — never real secrets on stage.

3.0 Preparation (10 minutes before)

1. Start Zuko (`npm run tauri dev`) and confirm Settings → Protection: Hooks and Gateway installed, Claude Code restarted.
2. Create a throw-away demo folder and the fake data file below. Open a terminal in it and start `claude`.
3. Optional: Ollama running with `gemma3:4b`; Local AI switched on; the browser extension loaded and linked.
4. Have Settings open on the **Activity** tab on a second screen so judges see each decision arrive.

```
mkdir -p ~/zuko-demo/build && cd ~/zuko-demo
cat > demo-secrets.txt <<'EOF'
Customer: Rahim Uddin, House 12, Road 5, Dhanmondi, Dhaka
Email: rahim.uddin@acme-demo.io Phone: +8801712345678
Card: 4242 4242 4242 4242
OPENAI_API_KEY=sk-proj-ZUKOTEST1234567890abcdefghijklmnopqrstuvwxyz
EOF
claude
```

All values above are fake (4242 4242 4242 4242 is a public test card number). The demo works without internet access for demos 7–11 if needed; demos 1–5 need Claude Code online.

Demo 1 · Meet Zuko

Shows: the guardian and his firebending reactions.

Steps

1. Point at the top-centre of the screen: the island opens.
2. Triple-click Zuko quickly.
3. Restart the app once to show the launch greeting (fire swirl entrance).

What the judges should see

- Zuko fire-punches a fireball at the cursor.
- The launch greeting: he drops in a fire swirl and his eyes ignite.

Demo 2 · Your secret never reaches the AI

Shows: masking in the prompt and local restore into a file (gateway mode).

Steps

1. In the Claude Code session, send the prompt below.
2. When it finishes, open the file it wrote.

```
My OpenAI key is sk-proj-ZUKOTEST1234567890abcdefghijklmnopqrstuvwxyz.
Create a file named .env here with one line: OPENAI_API_KEY=<the key>.
Then reply in one sentence.
```

What the judges should see

- The island shows a privacy notice (“Masked ... OpenAI API key”) and Zuko flicks fire at it.
- `.env` contains the **real** fake key.

- Settings → Activity shows a **Gateway · masked** row and the Write of .env; Settings → Vault lists `{{API_KEY_1}}`.

Ask Claude afterwards “What exactly is my key?” — it can only answer with the placeholder, because it never received the value.

Demo 3 · Proof: what the provider actually received

Shows: a recording of every request body that left the machine.

Steps

1. In a second terminal, start the headless gateway with a dump file (it prints its URL).
2. In a third terminal, run Claude Code once through that URL.
3. Count the key in the dump: zero. Count the placeholder: more than zero.

```
# terminal 2 (from Zuko/app)
export ZUKO_DATA_DIR=/tmp/zuko-proof
export ZUKO_GATEWAY_DUMP=/tmp/zuko-proof/sent.jsonl
cargo run --release --bin zuko-gateway
# -> prints http://127.0.0.1:<port>/t/<token>

# terminal 3 (any folder)
export ANTHROPIC_BASE_URL="http://127.0.0.1:<port>/t/<token>"
claude -p --model haiku "My key is sk-proj-ZUKOTEST1234567890abcdefghijklmnopqrstuvwxyz.
Repeat it back as KEY=<the key>."

grep -c ZUKOTEST /tmp/zuko-proof/sent.jsonl      # -> 0 (never sent)
grep -c API_KEY_1 /tmp/zuko-proof/sent.jsonl     # -> > 0 (placeholder sent instead)
```

What the judges should see

- Claude’s answer shows the real fake key (restored locally).
- `grep -c ZUKOTEST` on the dump prints **0**; `grep -c API_KEY_1` prints a positive number.

Some models politely refuse to repeat keys; that is the model's choice, not a Zuko failure. The dump still proves what was sent.

Demo 4 · Firewall: a banned website

Shows: policy enforcement on a shell command, with the reason fed back to the agent.

Steps

1. Send the prompt below in Claude Code.

Use the Bash tool to run exactly: `curl -s https://pastebin.com/raw/zukotest` and then quote the result word for word.

What the judges should see

- Zuko gets angry and fire-punches the blocked row; a red badge appears.
- Claude replies that the command was blocked because pastebin.com is on your blocked list.
- Settings → Activity: a **deny** row with rule `network.blocked:pastebin.com`.

Demo 5 · Approvals that make you look

Shows: risk scoring, plain-English headline, hold-to-allow.

Steps

1. Send the prompt below (the demo folder has an empty build/ folder).
2. Try a quick click on Allow, then press and hold it.

```
Delete the build folder in this directory with rm -rf build
```

What the judges should see

- A red **HIGH** card: "DELETES the folder build/ ..." with impact chips; Zuko's flame aura simmers.
- A quick click does nothing; holding fills the button for 1.5 s, then the action runs.
- Contrast: "run ls" passes with no card at all (low risk, auto-allowed).

Demo 6 · Works even when Zuko is closed

Shows: the relay enforces the same policy without the app.

Steps

1. Quit Zuko (tray → Quit; note: turn Gateway off first if Claude Code must keep working).
2. In Git Bash, from Zuko/app, pipe sample hook events into the relay.

```
cat > curl.json <<'EOF'
{"hook_event_name":"PreToolUse","session_id":"demo","cwd":"D:/demo",
 "tool_name":"Bash","tool_input":{"command":"curl https://pastebin.com/raw/x"}}
EOF
cat > ls.json <<'EOF'
{"hook_event_name":"PreToolUse","session_id":"demo","cwd":"D:/demo",
 "tool_name":"Bash","tool_input":{"command":"ls"}}
EOF
cat > prompt.json <<'EOF'
{"hook_event_name":"UserPromptSubmit","session_id":"demo","cwd":"D:/demo",
 "prompt":"my key is sk-proj-ZUK0TEST1234567890abcdefghijklmnopqrstuvwxyz"}
EOF
./target/release/zuko-hook.exe PreToolUse < curl.json           # -> deny (blocked list)
./target/release/zuko-hook.exe PreToolUse < ls.json             # -> nothing (no objection)
./target/release/zuko-hook.exe UserPromptSubmit < prompt.json  # -> block (Zuko not running)
```

What the judges should see

- The pastebin call prints a **deny**; ls prints nothing (no objection); the secret prompt prints a **block** saying Zuko is not running and the prompt was not sent.

Demo 7 · Documents: scrub before you upload

Shows: masking a whole file, including a PDF.

Steps

1. Drag demo-secrets.txt onto the island to show Zuko's fire scan.
2. Open Settings → **Documents** and drop the same file in the drop zone (or paste its full path).
3. Click **Show file** and open the generated demo-secrets.zuko.md.

What the judges should see

- Zuko sweeps fire down the page; secret lines burn into placeholder blocks.
- Findings list: OpenAI API key, email, phone, card — and, with Local AI on, the name and address ("AI deep scan: +N items").

- The .zuko.md contains only placeholders such as `{{API_KEY_1}}`, `{{CARD_1}}`.

Demo 8 · Clipboard: any chat app

Shows: masking and restoring text anywhere with hotkeys.

Steps

1. Copy the line below, press **Ctrl+Alt+M**, paste into Notepad.
2. Copy the masked line, press **Ctrl+Alt+U**, paste again.

```
my key is sk-proj-ZUKOTEST1234567890abcdefghijklmnopqrstuvwxyz and mail rahim@acme-demo.io
```

What the judges should see

- First paste: my key is `{{API_KEY_1}}` and mail `{{EMAIL_1}}`.
- Second paste: the original text, restored.

Demo 9 · Local AI finds what patterns miss (optional)

Shows: names and addresses detected by gemma3:4b on the laptop.

Steps

1. Settings → Local AI → status dot green → press **Test**.
2. Settings → Documents: drop demo-secrets.txt again (or, with Chat set to Local (Ollama), ask the island chat about it — no key needed and nothing leaves the PC).

What the judges should see

- Test result lists a **person name** and a **street address**.
- On a high-risk approval card, an “AI explanation” line appears a few seconds later.
- Say clearly: the AI can only add masks — it can never unblock or approve anything.

Demo 10 · Web chats: ChatGPT / claude.ai / DeepSeek

Shows: the browser extension masking prompts and restoring answers.

Steps

1. Open chatgpt.com (or claude.ai / chat.deepseek.com) with the extension loaded; open DevTools → Network.
2. Send: “My key is sk-proj-ZUKOTEST1234567890abcdefghijklmnopqrstuvwxyz — write a Python line that sets it as an env var.”
3. Click the conversation request in Network and look at its payload. Press **Alt+R** on the page.

What the judges should see

- The request payload contains `{{API_KEY_1}}`, not the key; a Zuko toast says it masked 1 item.
- The answer on screen shows the real key (highlighted); Alt+R toggles back to the placeholder.
- Uploading demo-secrets.txt is replaced by a masked .zuko.md.

Demo 11 · Records you can trust

Shows: audit integrity and safe vault tools.

Steps

1. Settings → **Activity** → **Verify audit log**.
2. Settings → **Vault** → **Inspect** on `{{CARD_1}}` and `{{PHONE_1}}`; then **Copy value**.

What the judges should see

- Verify shows **PASS** with the number of receipts.
- Inspect shows facts such as Brand: Visa · Last four: 4242 · Luhn check passes, and Country: Bangladesh (+880) · Operator: Grameenphone (mobile) — never the full value.
- Copy value: "clipboard clears in 30 s".

Demo 12 · Show the engineering

Shows: tests and the attack benchmark running live.

Steps

1. From Zuko/app run the commands below.

```
cargo test -p zuko-core --test scenarios -- --nocapture # attack benchmark + metrics
cargo test -p zuko-core                               # engine
cargo test -p zuko --lib                                # desktop app
cargo test -p zuko-hook && cargo test -p zuko-native-host # relay + browser bridge
cd ../extension && npm test                             # browser extension
```

What the judges should see

- The benchmark table: **51 scenarios · 100% detection · 0% false blocks**, decision latency in microseconds.
- All suites green (449 tests in total).

3.13 Backup: show every screen without a live agent

If the network fails on stage, the whole UI runs with realistic fake data in a browser:

```
cd Zuko/app && npm run dev # then open these in Chrome:
http://127.0.0.1:1420/index.html?mock=1&scene=fire-block
http://127.0.0.1:1420/index.html?mock=1&scene=approval-high
http://127.0.0.1:1420/index.html?mock=1&scene=privacy
http://127.0.0.1:1420/index.html?mock=1&scene=activity
http://127.0.0.1:1420/settings.html?mock=1
http://127.0.0.1:1420/dev/character-preview.html
```

Other scenes: overview, gateway, privacy-blocked, rubber-stamp, hold-demo, fire-flick, fire-click, fire-ring, approval-low / medium / critical.

3.14 A 5-minute judge run

Time	Demo	One line to say
0:00	1 · Meet Zuko	"A guardian that lives on your desktop and watches your AI agent."
0:30	2 · Secret in a prompt	"The model only ever sees {{API_KEY_1}} — my .env still gets the real key."
1:30	3 · Proof dump	"Here is every byte that left the machine: zero copies of the key."
2:15	4 · Banned website	"Rules the agent can't break — and it's told why."
2:45	5 · Hold-to-allow	"Risky actions need a hold; safe ones never interrupt you."
3:30	7 / 10 · Documents or web chat	"The same protection for files and ChatGPT."
4:15	12 · Benchmark	"51 attack scenarios: 100% caught, 0 false blocks, 0.3 ms per decision."